

Cahier des Charges — Projet de Fin d'Études
3ème Année Ingénieur — Génie Informatique Industriel

SmartMaintain

Plateforme de Maintenance Prédictive Industrielle

Basée sur l'Intelligence Artificielle et l'IoT Industriel

Étudiant	Mohamed Sadok Souguir
Encadrante	Mme. Imtinen Jemaa
Entreprise d'accueil	Photocarb
Établissement	ENETCOM — École Nationale d'Électronique et des Télécommunications de Sfax
Spécialité	Génie Informatique Industriel — 3ème Année Ingénieur
Année universitaire	2025 / 2026

TABLE DES MATIÈRES

1. Introduction et Contexte
2. Objectifs du Projet
3. Périmètre — Composants et Défauts
4. Architecture Générale
5. Phase 1 — Entraînement ML (Offline)
6. Phase 2 — Architecture Microservices
7. Phase 3 — Intégration IoT Réelle
8. Application Web — Fonctionnalités et Technologies
9. Fine-Tuning Machine Personnalisée
10. Diagrammes UML
11. Exigences Non Fonctionnelles
12. Base de Données
13. API REST — Spécification
14. Sécurité
15. Glossaire

FICHE D'IDENTITÉ DU PROJET

Titre : SmartMaintain — Plateforme de Maintenance Préventive Industrielle par ML & IoT

Entreprise d'accueil : Photocarb

Domaine : Industrie 4.0 — IoT — Machine Learning — Architecture Microservices — Web

Composants couverts : Moteur Électrique · Pompe Centrifuge · Compresseur d'Air · Échangeur de Chaleur

Défauts détectés : 8 défauts essentiels (2 par composant) à forte valeur ML

Datasets : Fournis directement par Photocarb

Stack technique : Python · Keras · Flask · React.js · PostgreSQL · MQTT · Docker

Durée estimée : 6 mois (PFE solo)

Réalisé par : Étudiant Ingénieur — niveau intermédiaire ML & Web

1. INTRODUCTION ET CONTEXTE

1.1 Contexte Industriel

Dans le secteur industriel moderne, les arrêts imprévus des machines représentent l'une des principales causes de pertes financières. Une panne non anticipée coûte entre 10 000 et 250 000 euros par heure selon le secteur et la criticité de l'équipement. La maintenance préventive traditionnelle basée sur des calendriers fixes présente deux problèmes antagonistes : elle est soit trop fréquente et gaspille des ressources, soit insuffisante et laisse passer des pannes.

L'Industrie 4.0 et la démocratisation des capteurs IoT permettent désormais de collecter en temps réel des données multi-dimensionnelles sur l'état des machines. Associée au Machine Learning, cette collecte permet de passer d'une maintenance réactive vers une maintenance prédictive intelligente, capable de détecter les anomalies deux à huit semaines avant la panne réelle.

1.2 Limite des Seuils Classiques

La détection par seuils fixes constitue aujourd'hui la méthode standard de surveillance industrielle. Elle déclenche une alerte lorsqu'un capteur dépasse une valeur prédéfinie. Bien que simple à implémenter, cette approche souffre de limitations fondamentales que ce projet vise à surmonter.

Comparaison : Seuils classiques vs Machine Learning

- **Moment de détection** — Seuils : après dépassement de valeur. ML : deux à huit semaines avant la panne.
- **Combinaison de signaux** — Seuils : un signal à la fois. ML : corrélation multi-capteurs simultanée.
- **Tendances temporelles** — Seuils : non visibles. ML : apprises automatiquement par le réseau LSTM.
- **Adaptation par machine** — Seuils : universels et figés. ML : fine-tuning personnalisé par équipement.
- **Faux positifs** — Seuils : nombreux. ML : réduits grâce au contexte opératoire appris.
- **Type de défaut** — Seuils : non identifié. ML : classifié avec niveau de criticité.

1.3 Problématique

Comment concevoir un système intelligent capable de détecter précocement les défauts de machines industrielles en combinant IoT temps réel, Machine Learning prédictif, et une architecture microservices scalable, accessible via une application web de supervision ?

2. OBJECTIFS DU PROJET

2.1 Objectif Principal

Développer SmartMaintain, une plateforme complète de maintenance prédictive industrielle qui détecte les défauts de quatre composants critiques avec une avance de deux à huit semaines sur la panne réelle. La plateforme s'appuie sur des modèles ML entraînés sur des datasets fournis par Photocarb et alimentés en production par des capteurs IoT réels.

2.2 Objectifs Spécifiques

Machine Learning

1. Entraîner quatre modèles ML spécialisés (un par composant) sur les datasets fournis par Photocarb.
2. Atteindre un F1-Score supérieur à 88 % pour chaque modèle.
3. Démontrer quantitativement la supériorité du ML sur les seuils classiques, en jours d'avance de détection.
4. Implémenter un mécanisme de fine-tuning permettant d'adapter les modèles à de nouvelles machines.

Système et Architecture

5. Concevoir une architecture microservices avec cinq services indépendants et isolés.
6. Intégrer un pipeline IoT complet via le protocole MQTT.
7. Garantir une latence de bout en bout inférieure à 500 ms, de la mesure capteur à l'alerte interface.
8. Conteneuriser l'ensemble de la solution avec Docker Compose pour un déploiement en une commande.

Application Web

9. Développer un dashboard de supervision temps réel en React.js avec WebSocket.
10. Implémenter un système d'alertes à quatre niveaux avec acquittement utilisateur.
11. Fournir une interface de fine-tuning accessible à un technicien non expert en ML.
12. Générer des rapports de maintenance exportables en PDF et Excel.

3. PÉRIMÈTRE — COMPOSANTS ET DÉFAUTS

3.1 Justification du Choix des Composants

Les quatre composants retenus ont été sélectionnés selon trois critères industriels objectifs : la fréquence des pannes dans les usines, le coût d'un arrêt non planifié, et la valeur ajoutée réelle du ML par rapport aux seuils simples. Ces composants représentent plus de 80 % des coûts de maintenance non planifiée dans l'industrie manufacturière mondiale. Les datasets correspondants ont été mis à disposition directement par Photocarb.

3.2 Composant 1 — Moteur Électrique

Le moteur électrique est présent dans 100 % des usines sans exception et représente à lui seul 70 % de la consommation électrique industrielle. Une panne non planifiée coûte entre 5 000 et 50 000 euros par heure selon le secteur.

Défauts retenus

- **Dégradation roulement** — responsable de 41 % des pannes moteur. Les capteurs utilisés sont un accéléromètre vibratoire trois axes. Le modèle LSTM entraîné sur la FFT du signal détecte les fréquences caractéristiques (BPFO, BPFI) dès le stade 1, soit deux à six semaines avant la panne, là où un seuil RMS ne réagit qu'au stade 4, deux à trois jours avant la casse.
- **Déséquilibre et désalignement** — responsable de 31 % des pannes. Le modèle combine le signal de vibration à $1\times$ la fréquence de rotation avec le signal de courant triphasé pour évaluer la gravité avec une avance d'une à trois semaines.

3.3 Composant 2 — Pompe Centrifuge

Avec 300 millions de pompes en service dans le monde, ce composant est critique en pétrochimie, traitement de l'eau et industrie pharmaceutique. Le coût d'une panne varie de 2 000 à 200 000 euros selon la taille et le secteur.

Défauts retenus

- **Cavitation** — détruit la pompe en quelques semaines si elle n'est pas détectée. La cavitation produit une signature vibratoire haute fréquence d'amplitude modérée, invisible pour un seuil global. Le modèle LSTM haute fréquence détecte cette signature en une à deux semaines d'avance.
- **Usure garniture mécanique** — cause 44 % des pannes pompe. La dégradation se traduit par une montée lente de la vibration basse fréquence sur plusieurs semaines, invisible par un seuil fixe qui déclenche trop tard. Avance de détection : deux à quatre semaines.

3.4 Composant 3 — Compresseur d'Air

Présent dans 90 % des usines manufacturières, le compresseur d'air est souvent appelé la quatrième énergie industrielle. Un arrêt provoque l'immobilisation de toute la ligne pneumatique, pour un coût de 20 000 à 100 000 euros par heure.

Défauts retenus

- **Usure soupapes** — cause 35 % des pannes compresseur. La dégradation se traduit par une perte progressive du rendement volumétrique, visible uniquement en corrélant pression, débit et consommation électrique simultanément. L'Autoencoder multivarié détecte cette déviation trois à six semaines avant la panne.
- **Problème refroidissement huile** — cause 45 % des arrêts d'urgence. Suivi par la température huile et air, l'Autoencoder détecte la dérive d'une à trois semaines avant le déclenchement thermique.

3.5 Composant 4 — Échangeur de Chaleur

Indispensable en pétrochimie, agroalimentaire, HVAC et énergie, le coût de l'encrassement des échangeurs atteint 2,5 milliards d'euros par an en Europe. Une panne se traduit par des pertes énergétiques de 15 à 30 % et un risque de contamination du produit.

Défauts retenus

- **Encrassement progressif** — représente 60 % des défauts échangeur. Le ΔT (différence de température entrée/sortie) diminue progressivement. Le modèle LSTM apprend la relation normale entre ΔT , débit et conditions opératoires, et détecte quand le ΔT mesuré est anormalement bas pour ces conditions, éliminant les faux positifs du seuil fixe. Avance : trois à huit semaines.
- **Fuite interne** — critique pour la sécurité et la qualité du produit. Détectée par l'analyse croisée des quatre températures entrée/sortie côté chaud et froid. Avance : une à deux semaines.

3.6 Synthèse des 8 Défauts

Le tableau ci-dessous récapitule les huit défauts couverts par SmartMaintain, avec les capteurs associés, le modèle ML retenu et l'avance de détection obtenue.

Composant	Défaut	Capteurs	Modèle ML	Avance
Moteur	Dégradation roulement	Vibration → FFT	LSTM Classifier	2 à 6 sem.
Moteur	Déséquilibre / Désalignement	Vibration + Courant	LSTM Classifier	1 à 3 sem.
Pompe	Cavitation	Vibration HF + Pression	LSTM HF	1 à 2 sem.
Pompe	Usure garniture mécanique	Vibration BF	LSTM	2 à 4 sem.
Compresseur	Usure soupapes	Pression + Temp + Courant	Autoencoder	3 à 6 sem.
Compresseur	Refroidissement huile	Température huile + air	Autoencoder	1 à 3 sem.
Échangeur	Encrassement progressif	ΔT + Débit	Régression LSTM	3 à 8 sem.
Échangeur	Fuite interne	Températures croisées	Régression LSTM	1 à 2 sem.

4. ARCHITECTURE GÉNÉRALE

4.1 Vue d'Ensemble

SmartMaintain est organisé autour de trois phases successives et d'une architecture microservices à quatre couches. Les trois phases correspondent au cycle de vie naturel du projet : entraînement offline, déploiement applicatif, puis intégration IoT réelle. Les quatre couches correspondent aux responsabilités techniques distinctes du système en production.

4.2 Les Trois Phases du Projet

- **Phase 1 — Entraînement ML Offline (Mois 1-2)** : travail data science pur sur les datasets Photocarb — prétraitement, feature engineering, entraînement et évaluation des quatre modèles.
- **Phase 2 — Application Microservices (Mois 3-4)** : développement des cinq microservices Flask et de l'application React. Les datasets Photocarb sont utilisés pour l'entraînement et le prétraitement des modèles ML.
- **Phase 3 — Intégration IoT Réelle (Mois 5)** : intégration des vrais capteurs ESP32 envoyant leurs mesures en temps réel via MQTT. Aucun autre service n'est modifié.

4.3 Les Quatre Couches

- **Couche 1 — Données** : en phase 2 et 3, cette couche reçoit les mesures des capteurs IoT réels via MQTT. Les datasets Photocarb sont dédiés à la phase d'entraînement offline des modèles ML.
- **Couche 2 — Moteur ML** : service Python dédié qui charge les quatre modèles entraînés au démarrage, applique le prétraitement identique à celui de l'entraînement, calcule le score d'anomalie, et expose les endpoints d'inférence et de fine-tuning.
- **Couche 3 — Backend API** : cinq microservices Flask indépendants (IoT, ML, Alertes, Auth, Gateway) conteneurisés avec Docker, communiquant via API REST et Redis Pub/Sub.
- **Couche 4 — Frontend Web** : application React.js avec quatre pages principales — dashboard, surveillance temps réel, alertes et interface de fine-tuning — mises à jour via WebSocket.

5. PHASE 1 — ENTRAÎNEMENT ML (OFFLINE)

5.1 Datasets D' ENTRAÎNEMENT

L'ensemble des données d'entraînement . Chaque dataset correspond à un composant industriel spécifique.

- Moteur : CWRU Bearing Dataset (~500 MB) — quatre états : normal + trois types de défauts à plusieurs charges.
- Pompe : Pump Sensor Dataset (~40 MB) — cinquante-deux capteurs, sept états de fonctionnement.
- Compresseur : Industrial Compressor Dataset (~20 MB) — pression, température, courant, cinq états.
- Échangeur : Modèle thermique de Fourier simulé en Python — génère des séries de ΔT avec encrassement et fuite paramétrables.

5.2 Pipeline de Prétraitement

Signal Vibration — Moteur et Pompe

13. Chargement du signal brut temporel (fréquence d'échantillonnage : 48 000 Hz pour le CWRU).
14. Découpage en fenêtres glissantes de 1 024 points avec chevauchement de 50 %.
15. Application de la FFT (Fast Fourier Transform) sur chaque fenêtre.
16. Extraction des features : RMS, Kurtosis, Crest Factor, Peak-to-Peak, fréquences BPFO/BPFI.
17. Normalisation Min-Max avec sauvegarde des paramètres (min, max par feature) dans `preprocessing_params.pkl`.

CRITIQUE : Les paramètres de normalisation calculés sur le dataset d'entraînement doivent être sauvegardés et réutilisés à l'identique en production. Tout recalcul sur les données IoT invalide les prédictions du modèle.

Signal Multi-capteurs — Compresseur

- Chargement des trois signaux : pression, température huile, courant.
- Alignement temporel à fréquence d'échantillonnage commune.
- Calcul de statistiques glissantes sur 60 secondes : moyenne, écart-type, pente.
- Normalisation standardisée (z-score) avec sauvegarde de la moyenne et de l'écart-type.

Signal Thermique — Échangeur

- Calcul du $\Delta T = T_{\text{entrée_chaud}} - T_{\text{sortie_chaud}}$ à chaque instant.
- Calcul du coefficient d'efficacité normalisée par le débit.
- Construction de séquences temporelles glissantes de 30 minutes.
- Normalisation et sauvegarde des paramètres.

5.3 Architecture des Modèles ML

LSTM Classifier — Moteur et Pompe

Architecture : LSTM 128 unités $\rightarrow$ LSTM 64 unités $\rightarrow$ Dense 32 $\rightarrow$ Softmax 4 classes.
Entrée : séquence de 50 fenêtres $\times$ 15 features. Sortie : quatre classes — Normal, Défaut 1, Défaut 2, Critique. Fonction de perte : Categorical Cross-Entropy. Optimiseur : Adam (lr = 0.001) avec réduction sur plateau. Dropout 0.3 entre couches LSTM. Maximum 100 epochs avec Early Stopping (patience = 10).

Autoencodeur Multivarié — Compresseur

Architecture encodeur-décodeur : LSTM 64 $\rightarrow$ 32 $\rightarrow$ décodeur 32 $\rightarrow$ 64 $\rightarrow$ reconstruction.
Entraîné uniquement sur les données normales (apprentissage non supervisé). Le score d'anomalie est l'erreur de reconstruction MSE normalisée entre 0 et 1. Le seuil de détection correspond au percentile 95 de l'erreur sur les données normales du jeu d'entraînement.

Régression LSTM — Échangeur

Architecture : LSTM 64 unités $\rightarrow$ Dense 32 $\rightarrow$ Dense 1 (sortie scalaire). Entrée : séquence de 30 minutes de ΔT + débit + conditions opératoires. Sortie : ΔT prédit dans 24 heures. Une anomalie est détectée lorsque l'écart entre le ΔT prédit et le ΔT réel dépasse un seuil adaptatif. Fonction de perte : MAE.

5.4 Métriques d'Évaluation

Chaque modèle est évalué sur une séparation train/validation/test de 70/15/15 %, avec validation croisée temporelle pour éviter la fuite de données.

- Précision (Precision) — taux de vraies anomalies parmi les alertes déclenchées — cible minimale : 90 %.
- Rappel (Recall) — taux de pannes réelles détectées — cible minimale : 88 %.
- F1-Score — moyenne harmonique précision/rappel — cible minimale : 88 %.
- AUC (courbe ROC) — performance globale du classificateur — cible : $> 0,92$.
- Comparaison ML vs seuil — gain en avance de détection chiffré en jours et semaines, documenté dans le mémoire.

6. PHASE 2 — ARCHITECTURE MICROSERVICES

6.1 Principe

L'application est découpée en cinq microservices indépendants, chacun dans son propre conteneur Docker, chacun responsable d'une seule responsabilité technique. Ils communiquent via l'API Gateway pour les appels synchrones et via Redis Pub/Sub pour les événements asynchrones. Docker Compose orchestre l'ensemble en une seule commande.

6.2 Service IoT — Port 5001

Point d'entrée des données capteurs dans le système. Il écoute le broker MQTT Mosquitto sur le topic usine/# et reçoit les mesures des capteurs IoT en temps réel. Les données sont validées, horodatées et normalisées avant d'être publiées sur le canal Redis sensor.data et persistées dans la table raw_measurements de PostgreSQL.

6.3 Service ML — Port 5002

Cerveau du système. Au démarrage, il charge les quatre modèles .h5 et le fichier preprocessing_params.pkl en mémoire. Il écoute le canal Redis sensor.data, applique le pipeline de prétraitement, exécute l'inférence et publie le score d'anomalie sur ml.predictions. Il gère également le fine-tuning : réception du CSV utilisateur, validation, gel des premières couches, ré-entraînement des couches supérieures, évaluation et sauvegarde du nouveau modèle en base.

6.4 Service Alertes — Port 5003

Ce service écoute le canal Redis ml.predictions et applique la logique de décision en quatre niveaux : Normal (score 0 à 0,3), Attention (0,3 à 0,6), Alerte (0,6 à 0,8), Critique (0,8 à 1,0). Il crée l'alerte en base de données, envoie la notification via WebSocket au frontend et enregistre l'événement dans les logs. L'acquittement d'une alerte par un technicien est tracé avec l'identifiant de l'utilisateur et l'horodatage.

6.5 Service Authentification — Port 5004

Gère les utilisateurs et les tokens JWT. Trois rôles sont définis : Administrateur (accès total), Technicien (surveillance et acquittement), Viewer (lecture seule). Les tokens ont une durée de validité de 24 heures et contiennent le rôle et l'identifiant de l'utilisateur. Les autres services appellent /auth/verify avant de traiter chaque requête.

6.6 API Gateway — Port 5000

Point d'entrée unique pour le frontend React. Il vérifie le JWT sur chaque requête via le Service Auth avant de router vers le service cible : `/api/iot/*` → Service IoT, `/api/ml/*` → Service ML, `/api/alerts/*` → Service Alertes. Il relaie également les événements WebSocket du Service Alertes vers le frontend et applique la configuration CORS.

6.7 Communication entre Services

- **Synchrone — API REST Flask** : pour les requêtes utilisateur de la Gateway vers les services.
- **Asynchrone — Redis Pub/Sub** : pour le flux de données temps réel IoT → ML → Alertes.
- **Temps réel — WebSocket (Flask-SocketIO)** : pour les mises à jour live de la Gateway vers le frontend.
- **Conteneurisation — Docker + Docker Compose** : orchestration des cinq services + PostgreSQL + Redis.

7. PHASE 3 — INTÉGRATION IOT RÉELLE

7.1 Architecture IoT

La phase 3 intègre de vrais capteurs physiques ESP32 envoyant leurs données via MQTT. Le changement est localisé uniquement dans le Service IoT — aucun autre service n'est modifié. L'architecture reste identique.

7.2 Matériel Utilisé

- Microcontrôleur ESP32 DevKit ou Raspberry Pi 4 — lecture des capteurs et publication des messages MQTT.
- Capteur de vibration ADXL345 ou MPU6050 (3 axes) — mesure de l'accélération vibratoire.
- Capteur de température DS18B20 ou PT100 — mesure de la température machine.
- Capteur de pression BMP280 ou MPX5700 — mesure de la pression de fluide.
- Capteur de courant SCT-013 (pince ampèremétrique) — mesure du courant moteur sans coupure de circuit.
- Broker MQTT Mosquitto hébergé sur serveur local — réception et distribution des messages.

7.3 Protocole MQTT et Topics

MQTT (Message Queuing Telemetry Transport) est le protocole standard de l'IoT industriel. Il fonctionne sur un modèle publication/abonnement : le capteur publie ses mesures sur un topic et le Service IoT, abonné à ce topic, reçoit les données automatiquement.

- usine/moteur/vibration — payload : machine_id, timestamp, valeur, unité.
- usine/moteur/courant — payload : machine_id, timestamp, valeur_A, valeur_B, valeur_C.
- usine/pompe/vibration — payload : machine_id, timestamp, valeur.
- usine/pompe/pression — payload : machine_id, timestamp, aspiration, refoulement.
- usine/compresseur/multi — payload : machine_id, timestamp, pression, temperature, courant.
- usine/échangeur/temperatures — payload : machine_id, timestamp, T_entree_chaud, T_sortie_chaud, T_entree_froid, T_sortie_froid, debit.

7.4 Cohérence du Prétraitement

Le prétraitement appliqué sur les données IoT en temps réel doit être strictement identique au prétraitement effectué lors de l'entraînement ML. Le fichier preprocessing_params.pkl sauvegardé en Phase 1 est chargé par le Service ML et appliqué à chaque mesure reçue. Tout écart invalide les prédictions.

8. APPLICATION WEB — FONCTIONNALITÉS ET TECHNOLOGIES

L'application web SmartMaintain est développée en React.js. Elle offre une supervision temps réel du parc machines via WebSocket, un système d'alertes à quatre niveaux, une interface de fine-tuning accessible à un technicien non expert, et des rapports exportables. Cette section décrit les pages de l'application ainsi que les choix technologiques retenus pour les implémenter.

8.1 Pages de l'Application

Dashboard Principal

Vue synthétique de l'état de l'ensemble du parc. Quatre cartes machine avec code couleur (vert / jaune / orange / rouge) affichent le score d'anomalie ML de chaque équipement, mis à jour chaque seconde. Un bandeau de KPIs (alertes actives, taux de disponibilité, dernière anomalie) et une timeline des dix derniers événements complètent la vue.

Surveillance Temps Réel

Page dédiée à un composant sélectionné. Les courbes des capteurs sont affichées en direct (Recharts LineChart) avec superposition de la courbe de référence du comportement normal. Une jauge circulaire représente le score ML [0-1] en temps réel. Lorsque le score dépasse 0,3 le type de défaut probable est identifié. L'historique du score sur 24 heures et l'export CSV des mesures sont également disponibles.

Alertes

Liste des alertes actives triées par criticité, avec horodatage, machine, type de défaut et score. Chaque alerte peut être acquittée par un Technicien ou Administrateur — l'identifiant et l'heure sont enregistrés. Un historique filtrable par machine, niveau et période est accessible, accompagné d'un graphique de distribution des alertes.

Fine-Tuning

Interface de personnalisation des modèles ML (détaillée au chapitre 9). L'utilisateur glisse-dépose un fichier CSV de l'historique machine, suit la progression epoch par epoch via WebSocket, compare les métriques du nouveau modèle avec le modèle de base, puis active le modèle si les résultats sont satisfaisants.

8.2 Stack Technologique Frontend

Les technologies suivantes ont été retenues pour répondre aux contraintes de temps réel, de supervision industrielle et d'accessibilité pour des utilisateurs non experts.

- **React 18 (framework UI)** — architecture par composants réutilisables (carte machine, jauge, graphique, alerte), hooks natifs (useState, useEffect, useReducer, useContext) pour la gestion d'état, et React Router v6 pour la navigation SPA avec protection des routes par rôle (PrivateRoute).
- **WebSocket via Socket.IO** — communication bidirectionnelle persistante entre le Service Alertes (Flask-SocketIO) et le frontend React. Les événements temps réel émis sont : score_update (score + niveau par machine), new_alert (nouvelle alerte), ack_update (acquiescement) et iot_status (état MQTT). Latence cible inférieure à 500 ms bout en bout.
- **Recharts (visualisation)** — bibliothèque construite sur React + D3.js. Utilisée pour les courbes de capteurs en fenêtre glissante (LineChart), la jauge du score d'anomalie (RadialBarChart), la superposition signal mesuré / signal normal (ComposedChart) et la distribution des alertes (BarChart). Tous les graphiques sont responsives via ResponsiveContainer.
- **Axios (client HTTP)** — intercepteurs de requête pour l'injection automatique du JWT, intercepteurs de réponse pour la gestion centralisée des erreurs (redirection login sur 401), et annulation des requêtes en cours lors de la navigation.
- **Context API + useReducer (état global)** — Context pour les données stables (utilisateur connecté, rôle, thème) ; useReducer pour les transitions d'état complexes de la liste d'alertes (ajout, acquiescement, filtrage). Cette approche évite la complexité de Redux pour ce niveau de projet.
- **react-dropzone + barre de progression WebSocket** — sur la page Fine-Tuning, la zone de glisser-déposer valide le CSV côté client avant envoi. La barre de progression se met à jour à chaque epoch via WebSocket, affichant epoch N/50, loss courante et ETA.
- **JWT en mémoire (sécurité)** — le token n'est jamais stocké dans localStorage (risque XSS). Il est conservé dans le Context React et disparaît à la fermeture de l'onglet. Un cookie httpOnly contenant le refresh token permet de renouveler la session de façon transparente.

9. FINE-TUNING MACHINE PERSONNALISÉE

9.1 Concept

L'utilisateur dispose de deux choix pour exploiter les modèles ML. Premier choix : utiliser directement le modèle déjà entraîné sur les datasets Photocarb, sans aucune configuration supplémentaire — le système est opérationnel immédiatement. Deuxième choix (optionnel) : lancer un fine-tuning pour adapter le modèle de base à une machine spécifique. Cette option est facultative ; l'utilisateur n'est pas obligé de fournir ses données. Au lieu d'entraîner un nouveau modèle from scratch, on réutilise les connaissances du modèle de base et on les affine avec un minimum de données de la machine cible.

Principe du Transfer Learning : les premières couches du modèle LSTM ont appris des patterns universels (tendances, oscillations, chocs) valables pour toutes les machines similaires. On gèle ces couches et on ré-entraîne uniquement les dernières couches sur les données spécifiques de la nouvelle machine.

9.2 Workflow depuis l'Interface Web

18. L'utilisateur se connecte avec un compte Administrateur.
19. Il accède à la page Fine-Tuning et clique sur Nouvelle Machine.
20. Il saisit les informations : nom, type (moteur/pompe/compresseur/échangeur), localisation.
21. Il uploade un fichier CSV contenant l'historique de fonctionnement (minimum 7 jours, recommandé 30 jours).
22. Le système valide automatiquement le format du CSV et vérifie la présence des colonnes requises.
23. Le système sélectionne automatiquement le modèle de base le plus adapté au type de machine.
24. L'utilisateur clique sur Lancer le Fine-Tuning.
25. Une barre de progression affiche l'avancement en temps réel (epoch par epoch via WebSocket).
26. À la fin, les métriques du nouveau modèle sont affichées côte à côte avec le modèle de base.
27. L'utilisateur valide le nouveau modèle pour l'activer sur cette machine.

9.3 Processus Technique

- Validation CSV : vérification des colonnes, types, plages de valeurs, taux de valeurs manquantes < 5 %.

- Prétraitement : application du même pipeline que Phase 1 avec les paramètres sauvegardés.
- Chargement du modèle de base .h5 correspondant au type de machine.
- Gel des couches : `model.layers[0].trainable = False`, `model.layers[1].trainable = False`.
- Ré-entraînement des couches 3 et 4 sur les nouvelles données (50 epochs maximum).
- Évaluation : calcul précision, rappel, F1 sur 20 % des données uploadées.
- Sauvegarde du nouveau modèle en base avec métadonnées, version et comparatif.

9.4 Format CSV Requis

Moteur

- Colonnes obligatoires : `timestamp`, `vibration_x`, `vibration_y`, `courant_A`.
- Colonnes optionnelles : `courant_B`, `courant_C`, `temperature`.

Pompe

- Colonnes obligatoires : `timestamp`, `vibration`, `pression_aspiration`, `pression_refoulement`.
- Colonnes optionnelles : `temperature`, `debit`.

Compresseur

- Colonnes obligatoires : `timestamp`, `pression_aspiration`, `pression_refoulement`, `temperature_huile`, `courant`.
- Colonnes optionnelles : `temperature_air`, `debit`.

Échangeur

- Colonnes obligatoires : `timestamp`, `T_entree_chaud`, `T_sortie_chaud`, `T_entree_froid`, `T_sortie_froid`, `debit`.
- Colonnes optionnelles : `pression_entree`, `pression_sortie`.

9.5 Données Minimales Requises

- Durée historique : minimum 7 jours continus, recommandé 30 jours avec variations de charge.
- Fréquence d'échantillonnage : minimum 1 mesure par seconde, recommandé 10 Hz pour les signaux vibratoires.
- État des données : fonctionnement normal uniquement en minimum, normal + quelques anomalies labélisées en recommandé.
- Valeurs manquantes : moins de 5 % en minimum, moins de 1 % en recommandé.

11. EXIGENCES NON FONCTIONNELLES

11.1 Performance

- Latence mesure → alerte (bout en bout) : inférieure à 500 ms.
- Rafraîchissement dashboard WebSocket : inférieur à 1 seconde.
- Temps de réponse API REST : inférieur à 200 ms au percentile 95.
- Temps d'inférence ML par mesure : inférieur à 50 ms.

11.2 Précision ML

- F1-Score minimum par modèle : 88 %.
- Taux de faux positifs maximum : 5 %.
- Taux de détection des pannes réelles : supérieur à 85 %.

11.3 Scalabilité

- Machines surveillées simultanément : 50 machines et plus.
- Capteurs simultanés : 500 capteurs et plus.

11.4 Disponibilité et Déploiement

- Uptime système : 99 % hors maintenance planifiée.
- Commande de démarrage unique : docker-compose up.

11.5 Sécurité

- Authentification JWT d'une durée de 24 heures avec rôles RBAC.
- Chiffrement de toutes les communications réseau avec TLS 1.3.

11.6 Compatibilité

- Navigateurs supportés : Chrome, Firefox, Edge (deux dernières versions majeures).
- Résolution minimale : 1366 × 768, interface responsive.

12. BASE DE DONNÉES

12.1 Principe

Chaque microservice possède sa propre base de données PostgreSQL pour garantir l'isolation et l'indépendance. Les cinq bases sont hébergées sur le même serveur PostgreSQL mais dans des schémas séparés. Redis est utilisé uniquement comme bus de messages asynchrones entre services.

12.2 Schéma — Service IoT

- **machines** : id, nom, type, localisation, statut, date_install — catalogue des machines surveillées.
- **capteurs** : id, machine_id, type, unité, freq_hz, actif — capteurs associés à chaque machine.
- **raw_measurements** : id, capteur_id, timestamp, valeur, qualité — mesures brutes horodatées.

12.3 Schéma — Service ML

- **models_registry** : id, machine_type, version, chemin_fichier, precision, rappel, f1, is_active — registre des modèles entraînés.
- **ml_predictions** : id, machine_id, timestamp, score, niveau, default_type, model_id — prédictions ML horodatées.
- **finetune_jobs** : id, machine_id, statut, csv_path, epochs, resultat_json, created_at — historique des fine-tunings.

12.4 Schéma — Service Alertes

- **alerts** : id, machine_id, niveau, message, score, default_type, acquittee, acquittee_par, acquittee_at, created_at.
- **alert_rules** : id, machine_id, seuil_attention, seuil_alerte, seuil_critique, actif.

12.5 Schéma — Service Auth

- **users** : id, email, password_hash, nom, role, actif, derniere_connexion.
- **sessions** : id, user_id, token_hash, expire_at, created_at.

13. SÉCURITÉ

13.1 Modèle RBAC

- **Administrateur** : accès total — CRUD machines, gestion utilisateurs, fine-tuning, configuration des modèles.
- **Technicien** : lecture dashboard, surveillance temps réel, acquittement des alertes.
- **Viewer** : lecture seule du dashboard et de la liste des alertes.
- **API Client** : accès programmatique aux endpoints définis via clé API.

13.2 Mesures de Sécurité

- Tokens JWT signés avec clé secrète, expiration 24 heures.
- Mots de passe hashés avec bcrypt (coût 12).
- Chiffrement TLS pour toutes les communications réseau.
- Validation et sanitisation de toutes les entrées (injection SQL, XSS).
- Rate limiting sur l'API Gateway : 100 requêtes par minute par utilisateur.
- Logs d'audit de toutes les actions critiques : fine-tuning, création machine, acquittement.
- Variables d'environnement pour toutes les clés secrètes — jamais codées en dur.
- Communication MQTT sécurisée avec authentification username/password.

14. GLOSSAIRE

Autoencoder — Réseau de neurones qui apprend à reconstruire son entrée. Utilisé pour détecter les anomalies en mesurant l'erreur de reconstruction sur des données inconnues.

BPFO / BPFi — Ball Pass Frequency Outer/Inner race — fréquences caractéristiques de défaut de roulement calculables à partir des dimensions géométriques du roulement.

CWRU — Case Western Reserve University Bearing Dataset — dataset de référence mondial pour la détection de défauts roulement par analyse vibratoire.

Docker Compose — Outil d'orchestration de conteneurs Docker permettant de démarrer l'ensemble des microservices avec une seule commande.

ΔT Dérivant — Diminution progressive de la différence de température entre entrée et sortie d'un échangeur, signe caractéristique d'un encrassement en cours.

F1-Score — Moyenne harmonique de la précision et du rappel. Métrique principale pour évaluer un classificateur ML sur des données déséquilibrées.

FFT — Fast Fourier Transform — transformée de Fourier rapide convertissant un signal temporel en spectre de fréquences, révélant les signatures de défauts.

Fine-Tuning — Ré-entraînement partiel d'un modèle pré-entraîné sur de nouvelles données spécifiques. Technique de Transfer Learning.

Kurtosis — Indicateur statistique mesurant les queues d'une distribution. Élevé lorsque le signal contient des chocs périodiques typiques d'un roulement dégradé.

LSTM — Long Short-Term Memory — type de réseau de neurones récurrent efficace pour analyser les séquences temporelles et mémoriser les dépendances à long terme.

Microservice — Architecture logicielle où chaque fonctionnalité est un service indépendant avec sa propre base de données, son propre processus et son propre conteneur.

MQTT — Message Queuing Telemetry Transport — protocole IoT léger basé sur publication/abonnement, standard de l'industrie pour les capteurs connectés.

RMS — Root Mean Square — valeur efficace d'un signal vibratoire. Indicateur simple mais insuffisant seul pour la détection précoce des défauts de roulement.

RUL — Remaining Useful Life — durée de vie utile restante d'un composant, estimée par le modèle ML à partir des tendances des scores d'anomalie.

Score d'Anomalie — Valeur entre 0 et 1 calculée par le modèle ML représentant le degré d'anormalité d'une mesure par rapport au comportement normal appris.

WebSocket — Protocole de communication bidirectionnelle persistante entre serveur et navigateur, utilisé pour les mises à jour temps réel sans polling HTTP.

